



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第三章

栈和队列



栈
队列
优先级队列



线性结构

**与表不同的是，它们都是限制
存取位置的线性结构**



§3.3 队列 (queue)



- 定义

- 队列是只允许在一端删除，在另一端插入的线性表
- 允许删除的一端叫做队头(*front*)，允许插入的一端叫做队尾(*rear*)。

- 特性

- 先进先出(*FIFO, First In First Out*)



3.3.1 队列的概念

抽象数据类型



```
template <class E>
```

```
class Queue {
```

```
public:
```

```
    Queue() { };           //构造函数
```

```
    ~Queue() { };         //析构函数
```

```
    virtual bool EnQueue(E x) = 0;
```

//进队列

```
    virtual bool DeQueue(E& x) = 0;
```

//出队列

```
    virtual bool getFront(E& x) = 0;
```

//取队头

```
    virtual bool IsEmpty() const = 0;
```

//判队列空

```
    virtual bool IsFull() const = 0;
```

//判队列满

```
};
```



队列

定义

一种操作受限的线性表，只能在队尾插入、在队头删除

特性：先进先出（FIFO）

术语：队头、队尾、空队列、队头元素、队尾元素

基本操作

创、销

增、删（入队、出队，只能在规定的—端进行）

查（获得队头元素，但不删除）

判空



队列的数组存储表示——顺序队列



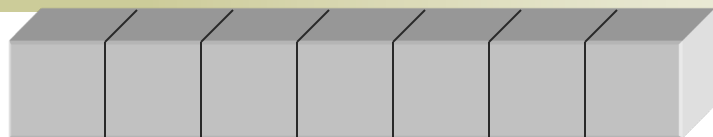
```
#include <assert.h>
#include <iostream.h>
#include "Queue.h"
template <class E>
class SeqQueue : public Queue<E> { //队列类定义
protected:
    int rear, front;                //队尾与队头指针
    E *elements;                    //队列存放数组
    int maxSize;                    //队列最大容量
public:
    SeqQueue(int sz = 10);          //构造函数
```



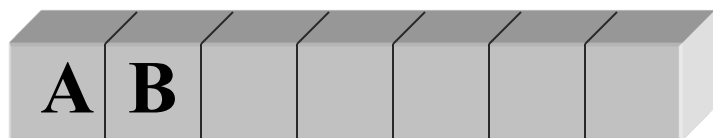
```
~SeqQueue() { delete[ ] elements; } //析构函数
bool EnQueue(E x);           //新元素进队列
bool DeQueue(E& x);          //退出队头元素
bool getFront(E& x);          //取队头元素值
void makeEmpty() { front = rear = 0; }
bool IsEmpty() const { return front == rear; }
bool IsFull() const
    { return rear ==maxSize; }
int getSize() const
    { return rear-front; }
};
```



队列的进队和出队 (数组方式)



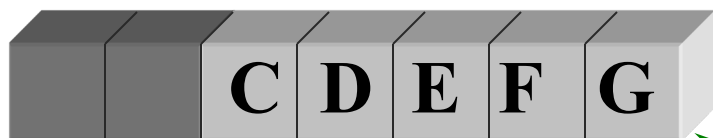
front rear 空队列



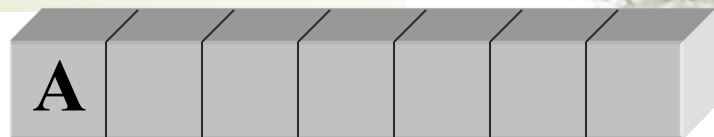
front rear B进队



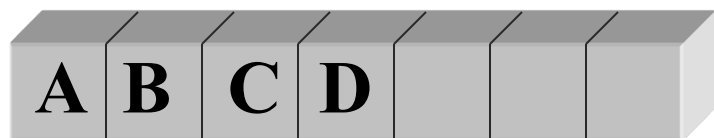
front rear A退队



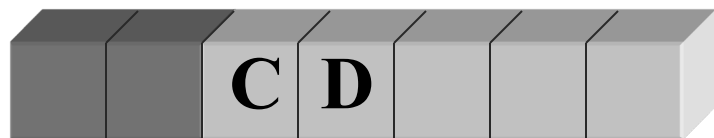
front rear E, F, G进队



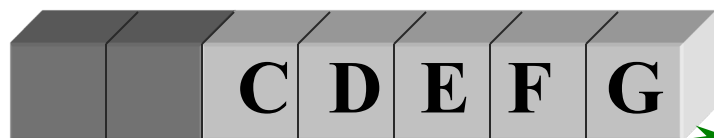
front rear A进队



front rear C, D进队



front rear B退队



front rear H进队, 溢出



队列的进队和出队



- 进队:

新元素在 $rear$ 处加入, $rear = rear + 1$

- 出队:

取出下标为 $front$ 的元素, $front = front + 1$

- 队空时 $rear == front$

- 队满时 $rear == maxSize$ (假溢出)

- 解决假溢出的办法之一: 将队列元素存放数组首尾相接, 形成循环 (环形) 队列。

- 用模运算将存储空间在逻辑上变成 “环状”

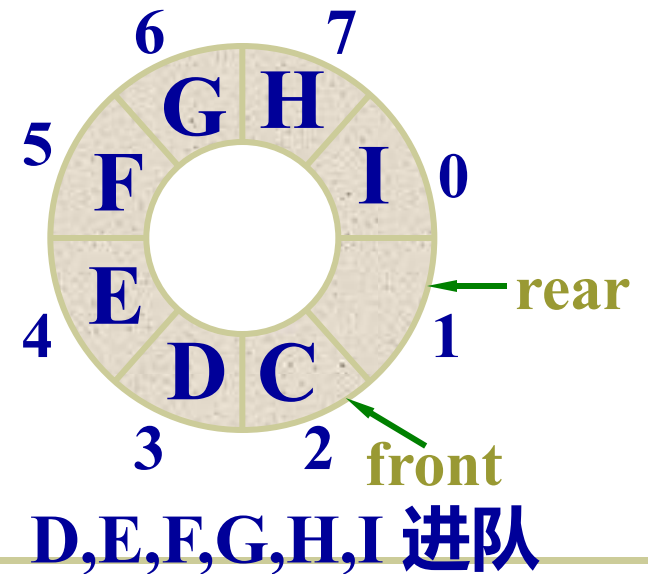
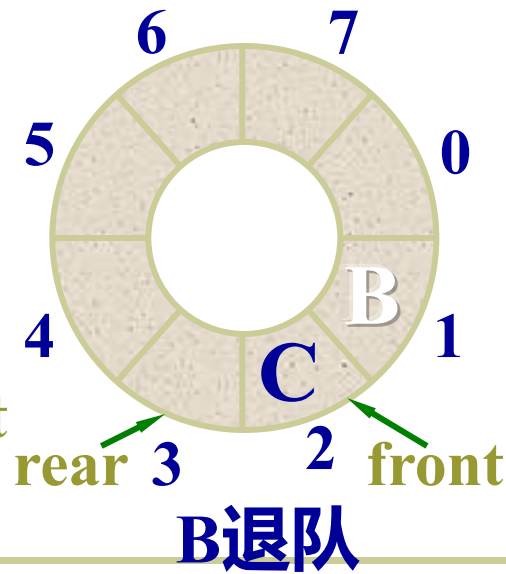
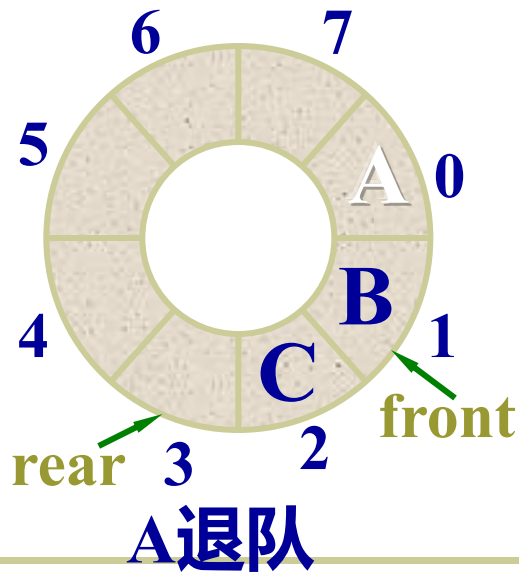
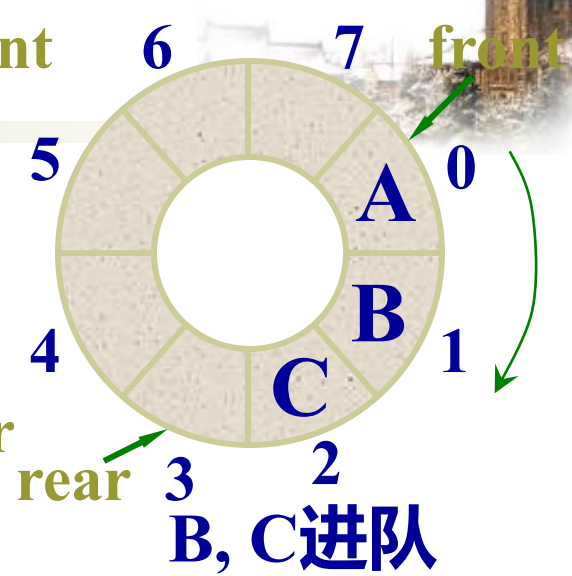
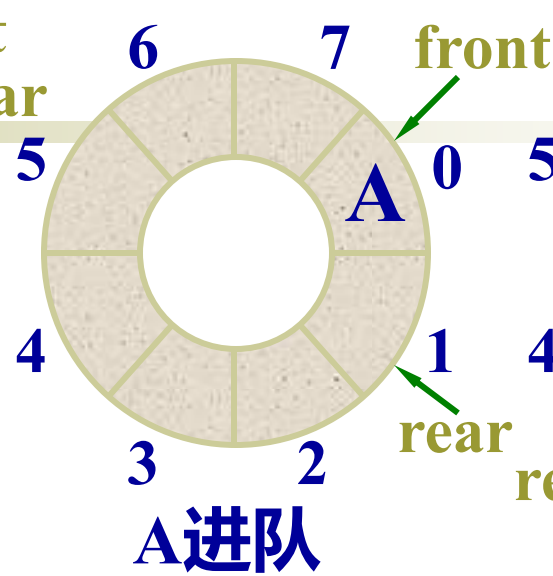
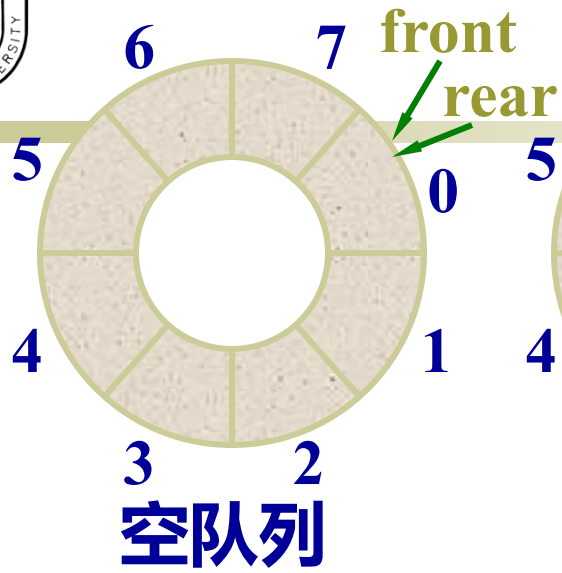


3.3.2 循环队列 (Circular Queue)



- 队列存放数组被当作首尾相连的表处理。
- 队头、队尾指针加1时从maxSize-1直接进到0, 可用语言的取模(余数)运算实现。
- 队头指针进1: $\text{front} = (\text{front} + 1) \% \text{maxSize};$
- 队尾指针进1: $\text{rear} = (\text{rear} + 1) \% \text{maxSize};$
- 队列初始化: $\text{front} = \text{rear} = 0;$
- 队空条件: $\text{front} == \text{rear};$
- 队满条件: $(\text{rear} + 1) \% \text{maxSize} == \text{front}$

这里为何要加1?





循环队列操作的定义



```
void MakeEmpty() { front = rear = 0; }  
int IsEmpty() const { return front == rear; }  
int IsFull() const  
    { return (rear+1) % maxSize == front; }
```

```
template <class E>  
SeqQueue<E>::SeqQueue(int sz)  
    : front(0), rear(0), maxSize(sz) { //构造函数  
    elements = new E[maxSize];  
    assert ( elements != NULL );  
};
```



```
template <class E>
```

```
bool SeqQueue<E>::EnQueue(E x) {
```

//若队列不满, 则将x插入到该队列队尾, 否则返回

```
if (IsFull() == true) return false;
```

```
elements[rear] = x;
```

//先存入

```
rear = (rear+1) % maxSize;
```

//尾指针加一

```
return true;
```

```
};
```



```
template <class E>
```

```
bool SeqQueue<E>::DeQueue(E& x) {
```

//若队列不空则函数退队头元素并返回其值

```
if (IsEmpty() == true) return false;
```

```
x = elements[front];           //先取队头
```

```
front = (front+1) % maxSize;    //再队头指针加一
```

```
return true;
```

```
};
```

```
template <class E>
```

```
bool SeqQueue<E>::getFront(E& x) const {
```

//若队列不空则函数返回该队列队头元素的值

```
if (IsEmpty() == true) return false; //队列空
```

```
x = elements[front];           //返回队头元素
```

```
return true;
```

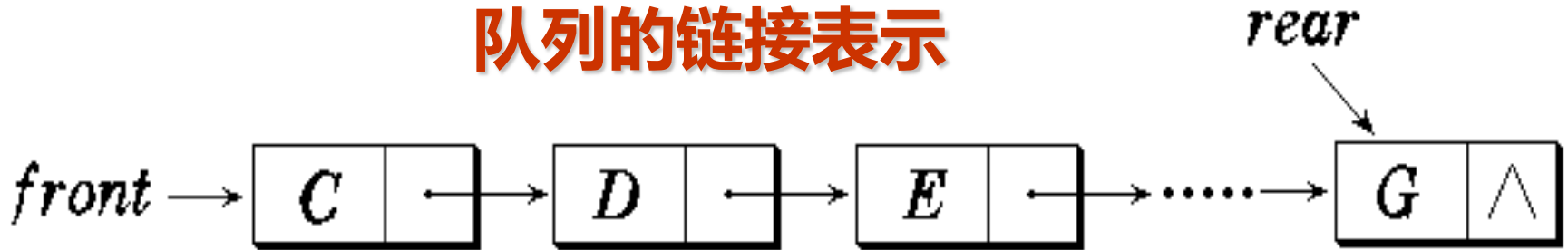
```
};
```



3.3.3 链式队列



队列的链接表示



- 队头在链头，队尾在链尾。
- 链式队列在进队时无队满问题，但有队空问题。
- 队空条件为 $front == NULL$



链式队列类的定义



```
#include <iostream.h>
```

```
#include "Queue.h"
```

```
template <class E>
```

```
struct QueueNode {
```

```
private:
```

```
    E data;
```

```
    QueueNode<E> *link;
```

```
public:
```

```
    QueueNode(E x = 0, QueueNode<E>
```

```
        *next = NULL) : data(x), link(next) { }
```

```
};
```

//队列结点类定义

//队列结点数据

//结点链指针



```
template <class E>
```

```
class LinkedQueue {
```

```
private:
```

```
    QueueNode<E> *front, *rear; //队头、队尾指针
```

```
public:
```

```
    LinkedQueue() : rear(NULL), front(NULL) { }
```

```
    ~LinkedQueue();
```

```
    bool EnQueue(E x);
```

```
    bool DeQueue(E& x);
```

```
    bool getFront(E& x);
```

```
    void makeEmpty(); //实现与~Queue()同
```

```
    bool IsEmpty() const { return front == NULL; }
```

```
};
```



```
template <class E>
```

```
LinkedQueue<E>::~~LinkedQueue() {//析构函数
```

```
    QueueNode<E> *p;
```

```
    while (front != NULL) { //逐个结点释放
```

```
        p = front; front = front->link; delete p;
```

```
    }
```

```
};
```



```
template <class E>
```

```
bool LinkedListQueue<E>::EnQueue(E x) {
```

//将新元素x插入到队列的队尾

```
if (front == NULL) { //创建第一个结点
```

```
    front = rear = new QueueNode<E> (x);
```

```
    if (front == NULL) return false; } //分配失败
```

```
else { //队列不空, 插入
```

```
    rear->link = new QueueNode<E> (x);
```

```
    if (rear->link == NULL) return false;
```

```
    rear = rear->link;
```

```
}
```

```
return true;
```

```
};
```



```
template <class E>
```

//如果队列不空，将队头结点从链式队列中删去

```
bool LinkedQueue<E>::DeQueue(E& x) {
```

```
    if (IsEmpty() == true) return false;    //判队空
```

```
    QueueNode<E> *p = front;
```

```
    x = front->data; front = front->link;
```

```
    delete p; return true;
```

```
};
```

```
template <class E>
```

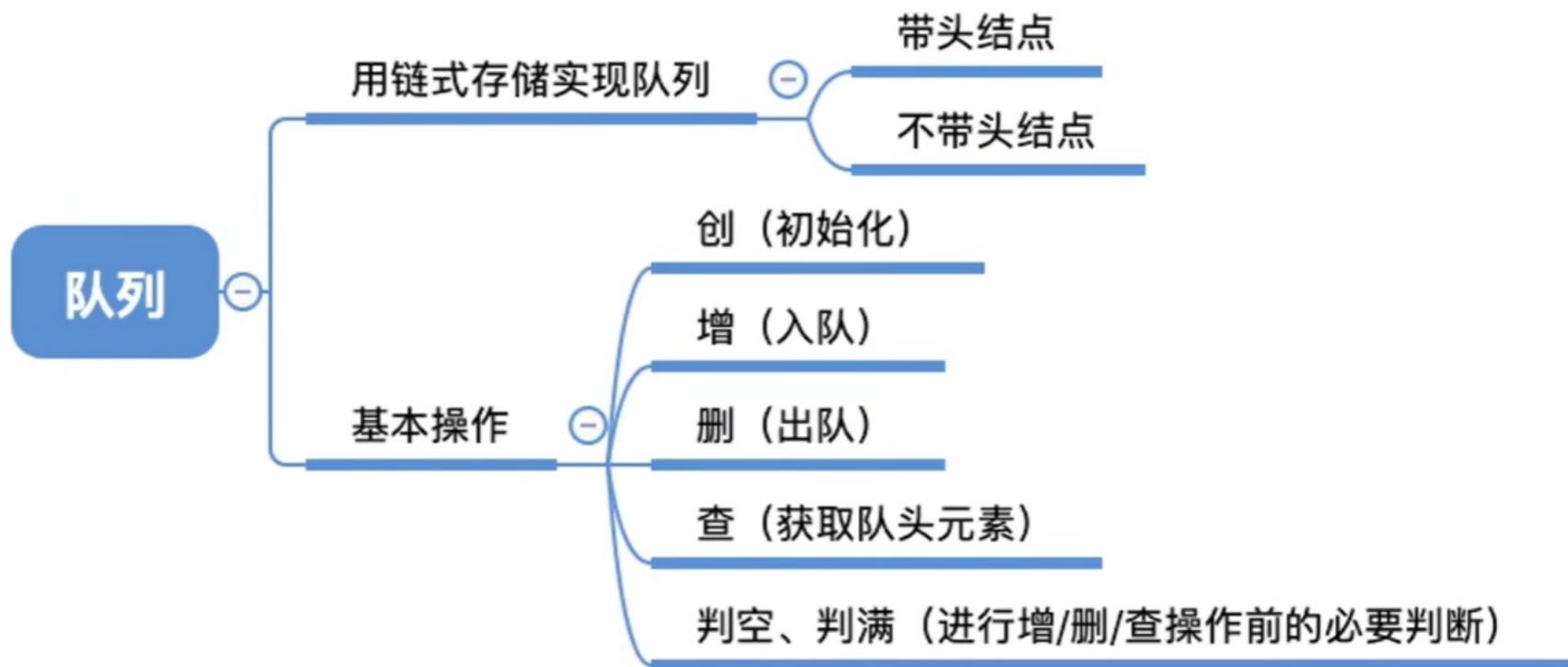
```
bool LinkedQueue<E>::getFront(E& x) {
```

//若队列不空，则函数返回队头元素的值

```
    if (IsEmpty() == true) return false;
```

```
    x = front->data; return true;
```

```
};
```





3.3.4 队列的应用



这个三角的两条斜边都是由数字1组成的，而其余的数都等于它肩上的两个数相加

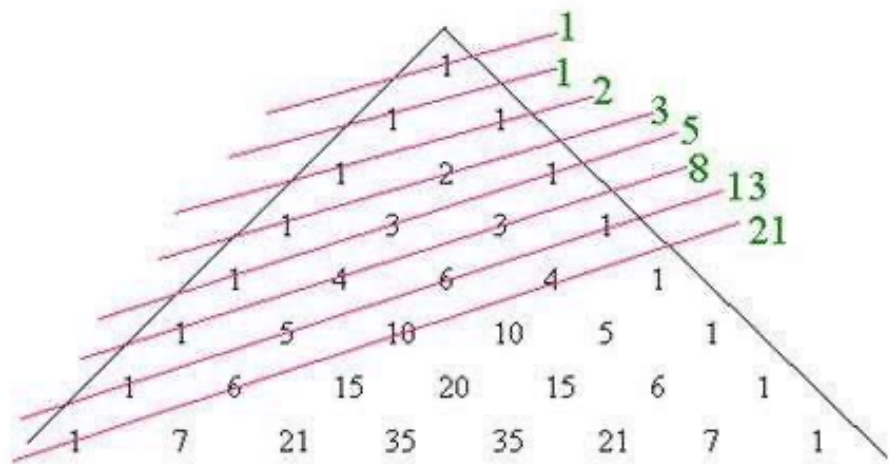
逐行打印二项展开式 $(a + b)^i$ 的系数

杨辉三角形 (Pascal's triangle)

将上的		1		1				$i = 1$				
相加		1		2		1		2				
		1		3		3		1		3		
		1		4		6		4		1	4	
		1		5		10		10		5	1	5
		1		6		15		15		6	1	6

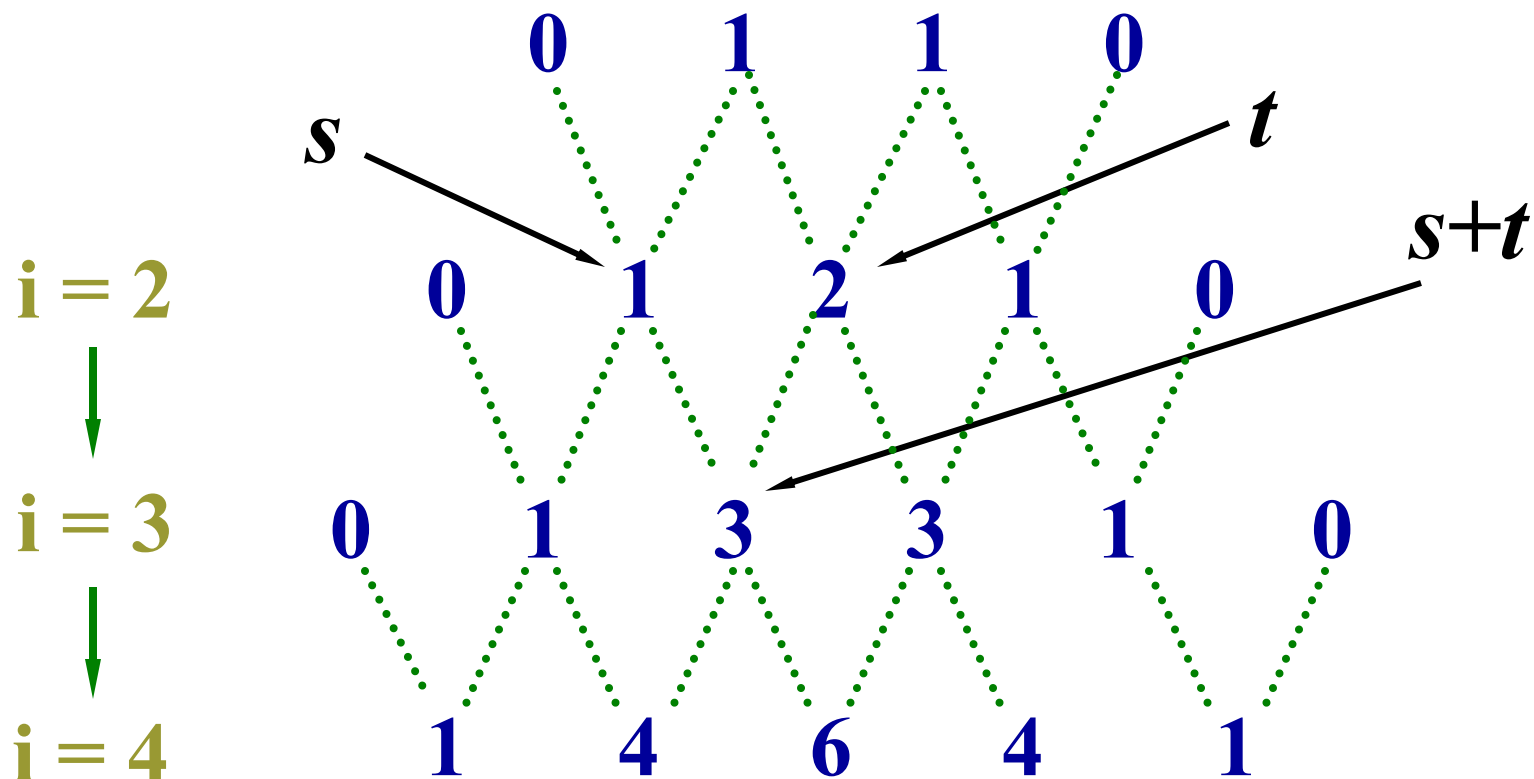


- 杨辉三角是公元1261年，我国宋代数学家杨辉在其著作《详解九章算法》中给出的一个用数字排列起来的三角形阵。
- 由于杨辉在书中引用了贾宪著的《开方作法本源》和“增乘开方法”，因此这个三角形也称“贾宪三角”。
- 在欧洲，这个三角形叫帕斯卡三角形，是帕斯卡在1654年研究出来的，比杨辉晚了近400年时间。





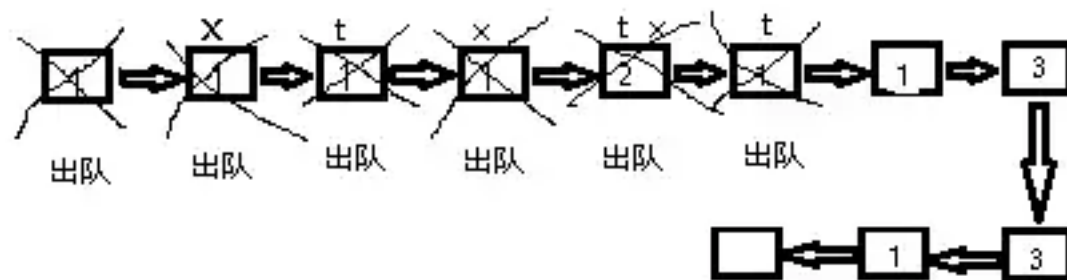
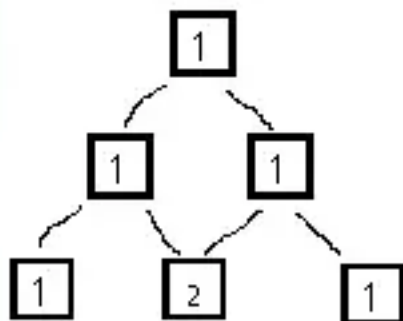
分析第 i 行元素与第 $i+1$ 行元素的关系



从前一行的数据可以计算下一行的数据



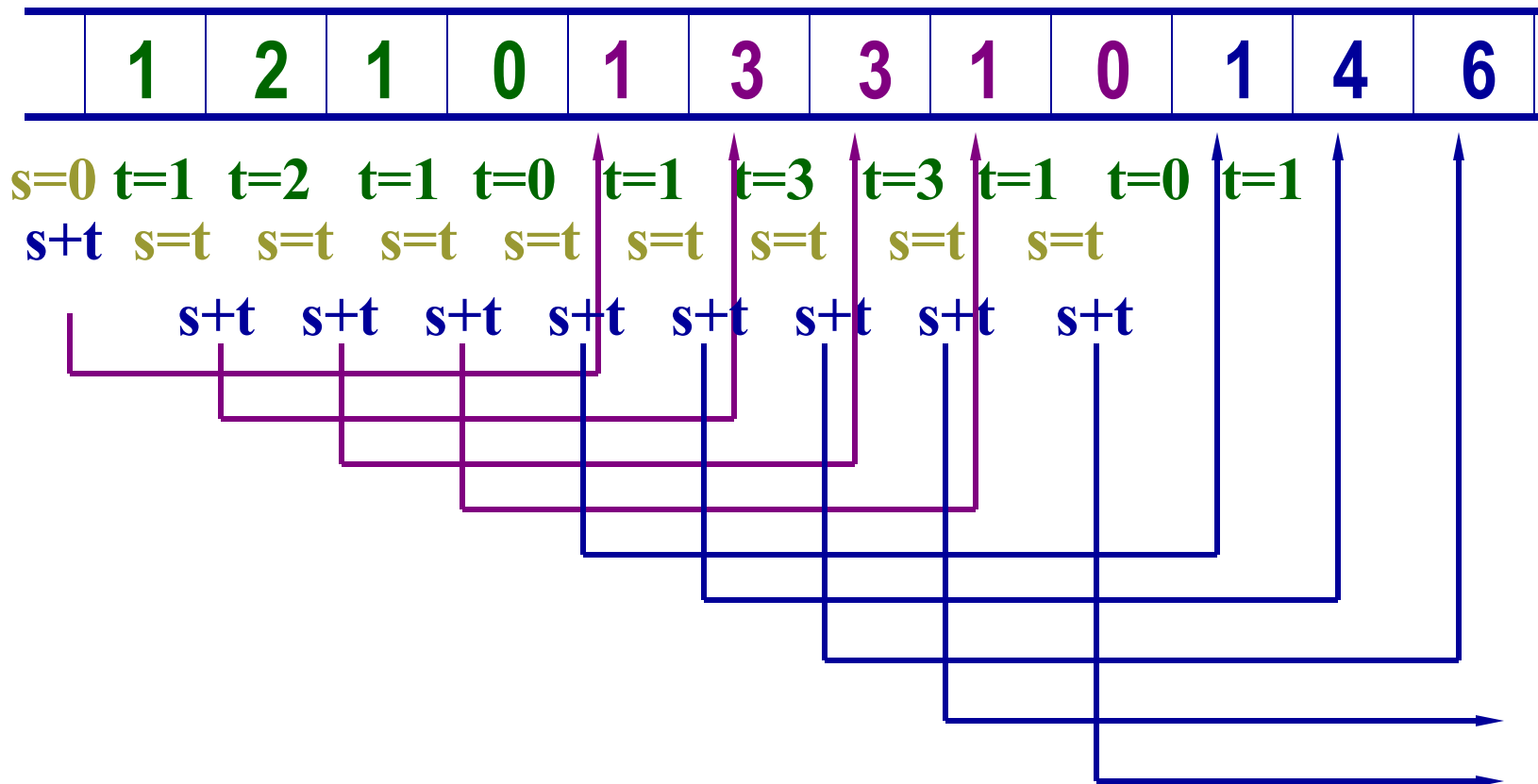
打印区



知乎 @五月



从第 i 行数据计算并存放第 $i+1$ 行数据





利用队列打印二项展开式系数的算法



```
#include <stdio.h>
#include <iostream.h>
#include "queue.h"
void YANGHVI(int n) {
    Queue q(n+1);           //队列初始化
    q.makeEmpty();
    q.Enqueue(1); q.Enqueue(1); //第一行的两个1
    int s = 0, t;
```



```
for (int i = 1; i <= n; i++) {           //逐行计算
    cout << endl;
    q.Enqueue(0);                         //各行间插入一个0
    for (int j = 1; j <= i+2; j++) {      //处理i+2个系数
        q.DeQueue(t);
        q.Enqueue(s + t);
        s = t;
        if (j != i+2) cout << s << ' ';
    }
}
}
```



实现方式	时间复杂度	空间复杂度	特点
队列实现	$O(n^2)$	$O(n)$	空间优化，只需存储一行数据
二维数组	$O(n^2)$	$O(n^2)$	直观易懂，但空间占用大



§3.4 优先级队列 (priority queue)



任务编号	1	2	3	4	5
优先权	20	0	40	30	10
执行顺序	3	1	5	4	2

数字越小，优先权越高

- **优先级队列**：是不同于先进先出队列的另一种队列，每次从队列中取出的是具有最高**优先权(优先级)**的元素



- 优先权是根据问题而定的
- 出现相同的优先权的元素时，按 *FIFO* 的方式处理



10	20	40	50	70	90				
----	----	----	----	----	----	--	--	--	--

↑ 插入 60

10	20	40	50	70	90	60			
----	----	----	----	----	----	----	--	--	--

60

10	20	40	50	60	70	90			
----	----	----	----	----	----	----	--	--	--

60

10	20	40	50	60	70	90			
----	----	----	----	----	----	----	--	--	--



优先级队列的类定义



```
#include <assert.h>
```

```
#include <iostream.h>
```

```
#include <stdlib.h>
```

```
template <class E>
```

```
class PQueue {
```

```
private:
```

```
    E *pqelements;
```

```
    int count;
```

```
    int maxPQSize;
```

```
    void adjust();
```

```
//存放数组
```

```
//队列元素计数
```

```
//最大元素个数
```

```
//调整
```



public:

```
PQueue(int sz = 50);  
~PQueue() { delete [ ] pquelements; }  
bool Insert(E x);  
bool removeMin(E& x);  
bool getFront(E& x);  
void makeEmpty() { count = 0; }  
bool IsEmpty() const { return count == 0; }  
bool IsFull() const  
    { return count == maxPQSize; }  
int Length() const { return count; }  
};
```



优先级队列部分成员函数的实现



```
template <class E>
```

```
PQueue<E>::PQueue(int sz) { //构造函数  
    maxPQSize = sz; count = 0;  
    pquelements = new E[maxPQSize];  
    assert (pquelements != NULL);  
}
```

```
template <class E>
```

```
bool PQueue<E>::Insert(E x) {  
    if (IsFull() == true) return false; //判队满断言  
    pquelements[count++] = x;           //插入  
    adjust(); return true;  
}
```



```
template <class E>
void PQueue<E>::adjust() {
    E temp = pquelements[count-1];
    //将最后元素暂存再从后向前找插入位置
    for (int j = count-2; j >= 0; j--)
        if (pquelements[j] <= temp) break;
        else pquelements[j+1] = pquelements[j];
    pquelements[j+1] = temp;
}
```



```
template <class E>
bool PQueue<E>::removeMin(E& x) {
    if (IsEmpty()) return false;
    x = pqelements[0];    //取出0号元素
    for (int i = 1; i < count; i++)
        pqelements[i-1] = pqelements[i];
        //从后向前逐个移动元素填补空位
    count--;
    return true;
}
```

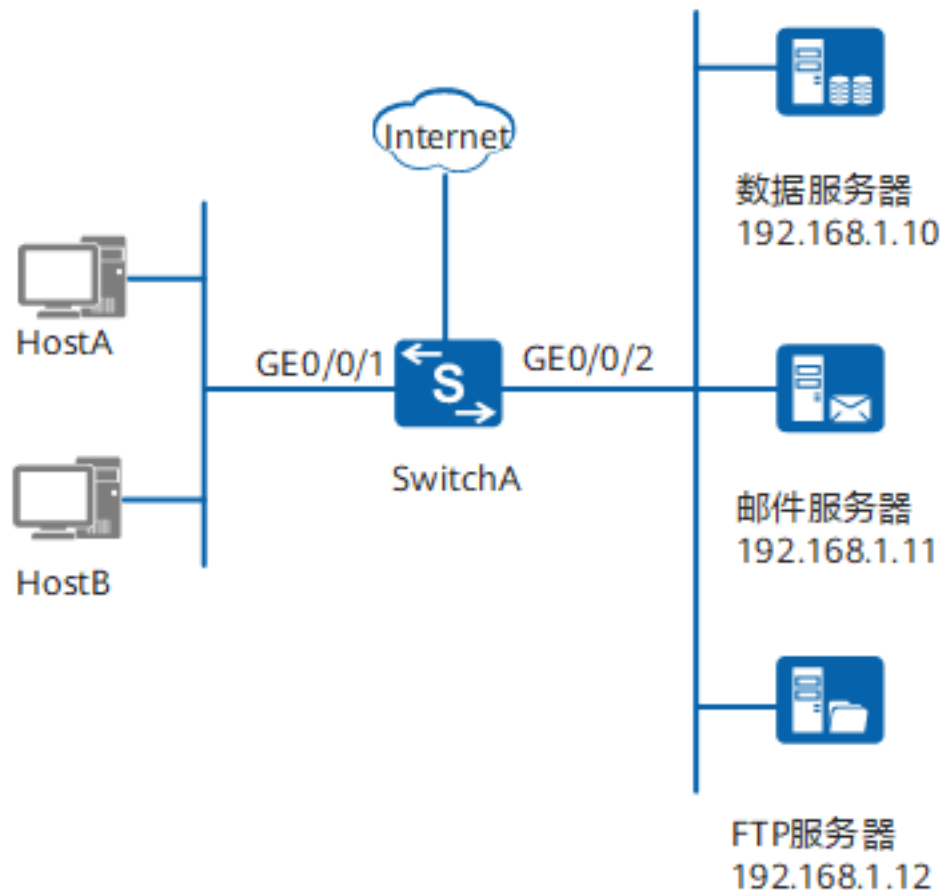


```
template <class E>
bool PQueue<E>::getFront (E& x) {
    if (IsEmpty() == true) return false;
    x = pquelements[0];
    return true;
}
```



- 优先队列是一种常见的数据结构,它可以用来存储具有**优先级**的元素。
- 这种数据结构在多种应用场景中都发挥着重要作用,如任务调度、路由算法、图形算法等。

- 某公司有数据查询、邮件处理、文件传输三种业务,由于业务的重要性不同,对业务处理的优先级也不同,公司用户HostA、HostB在访问三种业务服务器时,要求优先处理数据查询业务,其次为邮件处理,最后是文件传输。
- 通过在交换机上配置报文优先级标记和队列调度,保证数据查询业务优先级最高,其次是邮件处理,最后是文件传输。





优先级队列的几种实现方式



实现方式	插入时间复杂度	删除时间复杂度	空间复杂度
无序数组	$O(1)$	$O(n)$	$O(n)$
有序数组	$O(n)$	$O(1)$	$O(n)$
平衡BST	$O(\log n)$	$O(\log n)$	$O(n)$
堆	$O(\log n)$	$O(\log n)$	$O(n)$

堆的优势：

- 1.平均性能好：插入和删除都是 $O(\log n)$
- 2.空间效率高：可以用数组紧凑存储
- 3.缓存友好：数组存储有利于缓存命中
- 4.实现简单：相对其他平衡树结构更简单



栈: 只允许从一端插入和删除的线性表



队列: 只允许从一端插入、另一端删除的线性表



双端队列: 只允许从两端插入、两端删除的线性表



考点：判断输出序列的合法性

若数据元素输入序列为1, 2, 3, 4, 则下列哪些输出序列是合法的？
哪些是非法的？

可以是栈、队列或者双端队列

1234 1243 1324 1342 1423 1432
2134 2143 2314 2341 2413 2431
3124 3142 3214 3241 3412 3421
4123 4132 4213 4231 4312 4321



第3次书面作业



提交时间：9月25日，上午上课前提交

第3章：

- (1) 第131页，3.1
- (2) 第131页，3.2
- (3) 第131页，3.4：(4), (6)
- (4) 第132页，3.6